MODUL PRAKTIKUM 3

Searching: Sequential Search (Tanpa & Dengan Sentinel) dan Binary Search

Dosen Pengampu	Dr. Made Ayu Dusea Widyadara, M.Kom
Pertemuan	Ke-4 · Jumat, 9 Oktober 2026 · 13.00–14.40 · Ruang M41

A. Tujuan Praktikum

- Mahasiswa memahami pemanfaatan array dalam permasalahan komputasi, khususnya pencarian data.
- Mahasiswa mampu mengimplementasikan Sequential Search, baik tanpa sentinel maupun dengan sentinel.
- Mahasiswa mampu mengimplementasikan Binary Search pada data terurut dan menelusuri langkah kerjanya.
- Mahasiswa mampu membandingkan efisiensi sequential dan binary search berdasarkan jumlah perbandingan.

B. Alat dan Bahan

Kebutuhan	Keterangan
`Python 3.10+`	Sudah terpasang sejak Pertemuan 1, dengan akses melalui terminal
`Visual Studio Code`	Sudah terpasang beserta ekstensi Python (Microsoft)
`Folder kerja`	Buat folder baru pertemuan4/ di dalam folder kerja praktikum

C. Dasar Teori Ringkas

C.1 Konsep Pencarian

Pencarian (searching) adalah proses menemukan posisi sebuah nilai kunci (key) di dalam kumpulan data. Hasilnya berupa index jika ditemukan, atau -1 jika tidak ditemukan. Efisiensi pencarian diukur dari jumlah perbandingan yang dilakukan.

C.2 Sequential Search Tanpa Sentinel

Elemen diperiksa satu per satu dari index 0. Pada versi while, setiap putaran melakukan dua pemeriksaan: apakah index masih dalam batas (i < n) dan apakah data[i] sama dengan key. Data tidak perlu terurut. Kompleksitas O(n).

```
i = 0
while i < n and data[i] != key:
    i += 1
```

C.3 Sequential Search Dengan Sentinel

Key disalin ke ujung array sebagai sentinel, sehingga pencarian pasti berhenti dan pemeriksaan batas index tidak lagi diperlukan (cukup satu pemeriksaan per putaran). Jika berhenti di index n (posisi sentinel), berarti key tidak ada di data asli. Sentinel wajib dilepas kembali dengan pop() setelah pencarian.

C.4 Binary Search

Hanya dapat dipakai pada data terurut. Setiap langkah membandingkan key dengan elemen tengah, mid = (low + high) // 2. Jika data[mid] < key maka low = mid + 1; jika data[mid] > key maka high = mid - 1; perulangan berjalan selama low <= high. Kompleksitas O(log n).

D. Cara Menggunakan Modul Ini

- Contoh Soal — soal yang mirip/terkait, dibahas tuntas beserta kodenya. Pelajari polanya, boleh dijalankan, TIDAK dikumpulkan.
- Soal Latihan — soal sesungguhnya, WAJIB dikerjakan dan dikumpulkan. Tidak ada kode awal — susun program dari nol.

1. Buka folder pertemuan4/ di VS Code melalui File > Open Folder.
2. Buat file latihan1.py sampai latihan4.py untuk Soal Latihan.
3. Jalankan dengan tombol Run (▷), Ctrl+F5, atau ketik python latihan1.py di Terminal. Untuk program yang memakai input(), ketik nilainya di panel Terminal lalu tekan Enter.
4. Cocokkan hasil dengan "Format Output yang Diharapkan" untuk verifikasi mandiri.

LATIHAN 1

Sequential Search Dasar

Estimasi waktu: 20–25 menit

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah jasa ekspedisi menyimpan nomor resi paket yang sedang berada di gudang dalam array (tidak terurut). Petugas ingin mencari satu nomor resi, mengetahui posisinya, dan mencatat berapa banyak data yang sudah diperiksa sampai resi tersebut ditemukan.

Pembahasan

Pencarian dibungkus dalam fungsi sequential_search() yang mengembalikan dua nilai sekaligus: index hasil pencarian dan jumlah data yang diperiksa. Fungsi langsung return begitu data ditemukan; jika perulangan selesai tanpa kecocokan, fungsi mengembalikan -1.

```
resi = ["JNE1001", "JNT2002", "SCP3003", "POS4004", "JNE5005", "SCP6006"]
dicari = input("Masukkan nomor resi: ")

def sequential_search(data, target):
    diperiksa = 0
    for i in range(len(data)):
        diperiksa += 1
        if data[i] == target:
            return i, diperiksa
    return -1, diperiksa

posisi, diperiksa = sequential_search(resi, dicari)
if posisi != -1:
    print(f"Paket {dicari} ditemukan di index {posisi}")
else:
    print(f"Paket {dicari} tidak ditemukan")
print(f"Jumlah data yang diperiksa: {diperiksa}")
```

Jika pengguna memasukkan POS4004, program menampilkan: ditemukan di index 3, jumlah data yang diperiksa 4.

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Asisten praktikum menyimpan daftar NIM mahasiswa yang sudah melakukan presensi dalam array berikut:

```
presensi = ["2513020001", "2513020005", "2513020012", "2513020018",
            "2513020023", "2513020031", "2513020040"]
```

Buat program yang:

1. Meminta pengguna memasukkan NIM yang ingin dicek.
2. Menggunakan fungsi sequential_search(data, target) buatanmu sendiri yang mengembalikan index dan jumlah data yang diperiksa (tanpa memakai in atau index()).
3. Jika ditemukan, tampilkan urutan presensi (mulai dari 1) beserta index-nya; jika tidak, tampilkan bahwa NIM tidak terdaftar.
4. Selalu tampilkan jumlah data yang diperiksa.

Format Output yang Diharapkan (dua kali eksekusi)

```
Masukkan NIM yang dicari: 2513020023
NIM 2513020023 hadir, tercatat di urutan ke-5 (index 4)
Jumlah data yang diperiksa: 5

Masukkan NIM yang dicari: 2513020099
NIM 2513020099 tidak terdaftar dalam presensi
Jumlah data yang diperiksa: 7
```

Pertanyaan Reflektif

1. Mengapa jumlah data yang diperiksa selalu sama dengan panjang array ketika NIM tidak ditemukan?
2. Mengapa NIM disimpan sebagai string, bukan integer?

LATIHAN 2

Sequential Search Tanpa vs Dengan Sentinel

Estimasi waktu: 25–30 menit

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah toko online menyimpan kode voucher yang masih aktif. Buat dua versi fungsi pencarian — tanpa sentinel dan dengan sentinel — lalu bandingkan jumlah pemeriksaan (cek batas index + cek nilai) yang dilakukan masing-masing untuk kode yang ada dan kode yang tidak ada.

Pembahasan

Pada versi tanpa sentinel, setiap putaran menghitung dua pemeriksaan (i < n dan data[i] == target). Pada versi sentinel, target ditambahkan ke ujung array sehingga hanya perlu satu pemeriksaan per putaran. Setelah selesai, sentinel dilepas dengan pop() agar data asli tidak berubah.

```
voucher = ["HEMAT10", "GRATISONGKIR", "CASHBACK5", "DISKON20", "FLASH50"]

def seq_tanpa_sentinel(data, target):
    n = len(data)
    i = 0
    perbandingan = 0
    while True:
        perbandingan += 1            # cek i < n
        if i >= n:
            break
        perbandingan += 1            # cek data[i] == target
        if data[i] == target:
            break
        i += 1
    return (i if i < n else -1), perbandingan

def seq_dengan_sentinel(data, target):
    n = len(data)
    data.append(target)              # pasang sentinel
    i = 0
    perbandingan = 0
    while True:
        perbandingan += 1            # cek data[i] == target saja
        if data[i] == target:
            break
        i += 1
    data.pop()                       # lepas sentinel
    return (i if i < n else -1), perbandingan

for kode in ["DISKON20", "BONUS99"]:
    p1, c1 = seq_tanpa_sentinel(voucher, kode)
    p2, c2 = seq_dengan_sentinel(voucher, kode)
    print(f"Cari {kode}:")
    print(f"  Tanpa sentinel : index {p1}, {c1} perbandingan")
    print(f"  Dengan sentinel: index {p2}, {c2} perbandingan")
```

Hasil Eksekusi

```
Cari DISKON20:
  Tanpa sentinel : index 3, 8 perbandingan
  Dengan sentinel: index 3, 4 perbandingan
Cari BONUS99:
  Tanpa sentinel : index -1, 11 perbandingan
  Dengan sentinel: index -1, 6 perbandingan
```

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Sistem parkir sebuah kampus menyimpan plat nomor kendaraan pelanggan langganan bulanan:

```
parkir = ["AG1234AB", "AG5678CD", "L9012EF", "AG3456GH",
          "N7890IJ", "AG1111KL", "B2222MN", "AG9999OP"]
```

Buat program yang:

1. Mengimplementasikan dua fungsi: seq_tanpa_sentinel() dan seq_dengan_sentinel(), masing-masing mengembalikan index dan jumlah perbandingan (cek batas index dihitung sebagai perbandingan).
2. Mencari plat "AG1111KL" dan "AG0000ZZ" dengan kedua fungsi tersebut.
3. Menampilkan status plat (terdaftar beserta index, atau bukan pelanggan langganan) serta jumlah perbandingan dari kedua metode.
4. Di akhir program, tampilkan jumlah data parkir untuk membuktikan sentinel sudah dilepas (data harus tetap 8).

Format Output yang Diharapkan

```
Plat AG1111KL: terdaftar di index 5
  Tanpa sentinel : 12 perbandingan
  Dengan sentinel: 6 perbandingan
Plat AG0000ZZ: bukan pelanggan langganan
  Tanpa sentinel : 17 perbandingan
  Dengan sentinel: 9 perbandingan
Jumlah data setelah pencarian: 8
```

Pertanyaan Reflektif

1. Apa yang terjadi pada data parkir jika kamu lupa menulis data.pop() di fungsi sentinel, lalu pencarian dijalankan berkali-kali?

2. Jika data berisi n elemen dan key tidak ditemukan, rumuskan jumlah perbandingan untuk masing-masing metode (dalam n).

LATIHAN 3

Binary Search dengan Penelusuran Langkah

Estimasi waktu: 25–30 menit

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah toko bangunan menyimpan daftar harga barang yang sudah diurutkan dari termurah. Kasir ingin mencari apakah ada barang dengan harga tertentu menggunakan binary search, sambil menampilkan nilai low, high, dan mid di setiap langkah agar proses pencariannya terlihat.

Pembahasan

Rentang pencarian diwakili oleh low dan high. Di setiap langkah, mid dihitung dengan pembagian bulat (//). Karena data terurut, setengah bagian yang pasti tidak berisi key bisa langsung dibuang dengan menggeser low atau high.

```
harga = [15000, 25000, 30000, 45000, 60000, 75000, 90000, 120000, 150000]
dicari = 75000

def binary_search(data, target):
    low = 0
    high = len(data) - 1
    langkah = 0
    while low <= high:
        mid = (low + high) // 2
        langkah += 1
        print(f"Langkah {langkah}: low={low}, high={high}, "
              f"mid={mid}, data[mid]={data[mid]}")
        if data[mid] == target:
            return mid, langkah
        elif data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1, langkah

posisi, langkah = binary_search(harga, dicari)
print(f"Harga {dicari} ditemukan di index {posisi} "
      f"dalam {langkah} langkah")
```

Hasil Eksekusi

```
Langkah 1: low=0, high=8, mid=4, data[mid]=60000
Langkah 2: low=5, high=8, mid=6, data[mid]=90000
Langkah 3: low=5, high=5, mid=5, data[mid]=75000
Harga 75000 ditemukan di index 5 dalam 3 langkah
```

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Bagian pendaftaran sebuah rumah sakit menyimpan nomor rekam medis pasien yang sudah terurut:

```
rekam_medis = [1002, 1015, 1023, 1047, 1058, 1062, 1079, 1085,
               1091, 1104, 1118, 1125, 1137, 1149, 1156]
```

Buat program yang:

1. Membuat fungsi sudah_terurut(data) yang memeriksa apakah array sudah terurut naik. Jika belum, program menampilkan peringatan dan tidak menjalankan binary search.
2. Membuat fungsi binary_search(data, target) yang menampilkan nilai low, high, mid, dan data[mid] di setiap langkah.
3. Mencari nomor 1118 dan 1050, lalu menampilkan hasil akhir beserta jumlah langkah untuk masing-masing.

Format Output yang Diharapkan

```
Mencari rekam medis 1118:
Langkah 1: low=0, high=14, mid=7, data[mid]=1085
Langkah 2: low=8, high=14, mid=11, data[mid]=1125
Langkah 3: low=8, high=10, mid=9, data[mid]=1104
Langkah 4: low=10, high=10, mid=10, data[mid]=1118
Ditemukan di index 10 setelah 4 langkah

Mencari rekam medis 1050:
Langkah 1: low=0, high=14, mid=7, data[mid]=1085
Langkah 2: low=0, high=6, mid=3, data[mid]=1047
Langkah 3: low=4, high=6, mid=5, data[mid]=1062
Langkah 4: low=4, high=4, mid=4, data[mid]=1058
Rekam medis 1050 tidak ditemukan setelah 4 langkah
```

Pertanyaan Reflektif

1. Pada pencarian 1050, perhatikan nilai low dan high setelah langkah 4. Mengapa perulangan berhenti di sana?
2. Ubah satu angka di rekam_medis agar urutannya rusak (mis. tukar 1047 dan 1085). Apa yang ditampilkan programmu, dan mengapa pemeriksaan ini penting?

LATIHAN 4

Studi Kasus — Memilih Algoritma Pencarian

Estimasi waktu: 30–35 menit

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah warung yang menerima pembayaran QRIS ingin mencari SEMUA transaksi dengan nominal tertentu dalam catatan harian (data tidak terurut), lalu menghitung jumlah dan total nilainya. Karena yang dicari adalah semua kemunculan pada data acak, sequential search adalah pilihan yang tepat.

Pembahasan

Berbeda dengan pencarian biasa, perulangan tidak dihentikan saat data pertama ditemukan. Setiap index yang cocok disimpan ke dalam list posisi_ditemukan, sehingga di akhir program diketahui seluruh posisi transaksi tersebut.

```
transaksi = [25000, 50000, 15000, 50000, 75000, 50000, 20000]
dicari = 50000

posisi_ditemukan = []
for i in range(len(transaksi)):
    if transaksi[i] == dicari:
        posisi_ditemukan.append(i)

if posisi_ditemukan:
    print(f"Nominal Rp{dicari} ditemukan di index: {posisi_ditemukan}")
    print(f"Jumlah transaksi: {len(posisi_ditemukan)}")
    print(f"Total nilai: Rp{dicari * len(posisi_ditemukan)}")
else:
    print(f"Tidak ada transaksi bernilai Rp{dicari}")
```

Hasil Eksekusi

```
Nominal Rp50000 ditemukan di index: [1, 3, 5]
Jumlah transaksi: 3
Total nilai: Rp150000
```

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Sebuah koperasi simpan pinjam memiliki 1.000 anggota dengan nomor anggota yang sudah terurut. Buat datanya dengan perintah berikut (tidak perlu diketik manual):

```
anggota = list(range(10001, 12001, 2))   # 10001, 10003, ..., 11999
```

Buat program yang:

1. Menampilkan jumlah anggota koperasi.
2. Mengimplementasikan sequential_search() dan binary_search(), masing-masing mengembalikan index dan jumlah perbandingan.
3. Mencari nomor 11501 dan 10002 menggunakan kedua algoritma.
4. Menampilkan status nomor (ditemukan beserta index, atau tidak terdaftar) dan jumlah perbandingan dari kedua algoritma.
5. Menutup program dengan satu baris kesimpulan tentang algoritma yang lebih efisien untuk kasus ini.

Format Output yang Diharapkan

```
Jumlah anggota koperasi: 1000
Nomor 11501: ditemukan di index 750
  Sequential search: 751 perbandingan
  Binary search    : 9 perbandingan
Nomor 10002: tidak terdaftar
  Sequential search: 1000 perbandingan
  Binary search    : 10 perbandingan
Kesimpulan: binary search jauh lebih cepat untuk data besar yang sudah terurut.
```

Pertanyaan Reflektif

1. Contoh Soal memakai sequential search, sedangkan Soal Latihan cocok memakai binary search. Apa perbedaan kondisi data dan kebutuhan di kedua kasus yang membuat pilihannya berbeda?
2. Jika koperasi hanya mencari satu nomor sekali saja, tetapi datanya belum terurut, apakah tetap layak mengurutkan data dulu lalu memakai binary search? Jelaskan.

E. Ketentuan Pengumpulan

- Kumpulkan 4 file Soal Latihan: latihan1.py, latihan2.py, latihan3.py, latihan4.py (kode Contoh Soal tidak perlu dikumpulkan).
- Sertakan tangkapan layar (screenshot) hasil eksekusi tiap file di terminal VS Code.
- Kumpulkan sebelum Pertemuan 5 (Rabu, 14 Oktober 2026) melalui kanal yang ditentukan dosen.

F. Rubrik Penilaian

Aspek	Bobot
Kebenaran hasil eksekusi (output sesuai format yang diharapkan)	35%
Ketepatan implementasi algoritma (sequential, sentinel, binary)	30%
Kemandirian menyusun solusi (bukan menyalin pola Contoh Soal apa adanya)	15%
Kerapian & kejelasan kode (nama variabel, komentar)	10%
Jawaban pertanyaan reflektif	10%

Penilaian: kualitatif, tugas individu — mengacu pada RPS Struktur Data Sub-CPMK3 (bobot 10%).